

★ Column Transformer in sklearn

Think of ColumnTransformer as a tool to apply different preprocessing steps to different columns of our dataset.

In real datasets:

- Numeric columns → scaling
- Categorical columns → encoding
- Text columns → vectorization

Instead of preprocessing everything separately, Column Transformer does all at once in a clean pipeline.

Why do we need Column Transformer?

→ Problem:

- Some columns need StandardScaler
- Some need OneHotEncoder
- Some may be ignored

(If we apply everything manually → messy code)

Solution:-

→ ColumnTransformer handles this automatically

Syntax:

from sklearn.compose import ColumnTransformer

ColumnTransformer (

transformers = [

], ("name", transformer, column_list)

remainder = "drop" # or "passthrough"

Parameters

- name → label for transformer
- transformer → preprocessing method
- column_list → columns to apply on
- remainder

↳ drop → remove other columns
↳ passthrough → keep other columns

Date _____
Page _____

Example 1: - Basic Understanding

Date _____
Page _____

Dataset

Age	Salary	Gender	City
-----	--------	--------	------

- Age, Salary → numeric
- Gender, City → categorical

Step 1: - Import

import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

Step 2: - Data

df = pd.DataFrame ({

"Age": [25, 30, 35],
"Salary": [40000, 50000, 60000],
"Gender": ["M", "F", "M"],
"City": ["Kathmandu", "Pokhara", "Kathmandu"]
})

3. - Create ColumnTransformer

ct = ColumnTransformer(~~transformers~~)

("scale", StandardScaler(), ["Age", "Salary"]

"encode", OneHotEncoder(), ["gender", "city"]

), remainder = "drop")

Step 4: - fit & transform

result = ct.fit_transform(df)
print(result)

Q.P (approx)

([-1.22 -1.22 1 0 1 0]
[0 0 0 1 0 1]
[1.22 1.22 1 0 1 0]

What happened?

- Age, Salary → scaled
- Gender, city → one-hot encoded
- combined into one NumPy array

Example 2 - Keep other columns

If we want to keep remaining columns:

ct = ColumnTransformer(^{transformers}
("scale", StandardScaler(), ["Age"])
, remainder = "passthrough")

Now, only Age scaled, others kept same.

Using with Pipeline (Very Important)

Usually used inside pipeline.

from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

pipe = Pipeline(
 ("preprocessor", ct),
 ("model", LogisticRegression())
)

pipe.fit(X_train, y_train)

Benefit:-

- Entire preprocessing + model = single object
- No manual steps needed

When to use Column Transformer?

Use when:

- mixed data types
- ML model needs numeric input
- want clean pipeline
- avoid manual preprocessing

Quick Summary

Column Transformer:

- applies different preprocessing to different columns
- works with pipeline
- keeps code clean
- saves time
- very important for ML projects

Most real-world ML pipelines

Input as dataframes.

Input in

Dataframe format is required:-

If we use

Column Transformer

↳ which needs column's names to apply different transformations/preprocessing for different columns.

Thus, we need

input = pd.DataFrame ([&

"column 1" : "value 1"

"column 2" : "value 2"

] &)

model = predict (input) [0]

return list, and take only list's first value.

But for simpler model, no need of ~~probability~~

`model = Random Forest Classifier ()`

`model.predict (CC )`

Date _____
Page _____

`model.classes_`

What possible output labels does my trained classifier know?
`model.classes_` is an attribute in scikit-learn classification models that shows:

All the unique class labels the model has learned during training, in sorted order.

Where does it come from?

After you train (fit) a classifier from scikit-learn, it stores the class labels internally.

Example classifiers:-

- Logistic Regression
- Decision Tree Classifier
- Random Forest Classifier
- SVC

What exactly does it store?

⇒ It stores:-

`sorted (unique (y train))`

So, whatever labels you trained on - it keeps them